

Rapport d'analyse Technique & Dette : Service auth-api

Projet : Api d'authentification de <Votre Entreprise>

Langage : Java

Date : 14 août 2026

1. Synthèse Exécutive

L'analyse du code de l'API d'authentification montre une excellente base globale sur la maintenabilité (Note A) et la fiabilité (Note A), mais révèle deux vulnérabilités critiques de sécurité (Note D) qui bloquent le déploiement sécurisé en production.

Score de santé :

Sécurité : D (2 problèmes)

Fiabilité : A (0 bug)

Maintenabilité : A (2 Code Smells)

Couverture de Tests : 64,1 % (JaCoCo)

Duplication de Code : 0 %

- Risque Majeur de sécurité : Utilisation d'un algorithme de chiffrement obsolète (*cipher*) au niveau du module d'authentification. En cas d'attaque, des jetons ou mots de passe pourraient être interceptés/déchiffrés. Également, deux utilisateurs de test sont automatiquement déployés au lancement de l'application via la configuration, ce qui représente une faille de sécurité.
- Impact Business : Vulnérabilités critiques sur le service d'authentification. Une remise aux normes est requise avant toute ouverture aux utilisateurs.
- Effort de correction estimé : 2 jours
 - Correctifs de sécurité
 - montée de la couverture de tests global avec accent sur la config.

2. Diagnostic Technique

A. Sécurité (Niveau de Priorité : P0 - Critique)

- Score SonarQube : D
- Problème identifié : 2 issues critiques de sécurité ouvertes.
 - Emploi d'un algorithme de chiffrement faible/obsolète (*Cipher*).
Localisation : Package com.gatcha.auth_api.config.
Recommandation : Remplacer le chiffrement obsolète par un standard robuste.
 - Implémentation d'utilisateurs de test au lancement de l'application.
Localisation : Package com.gatcha.auth_api.config.
Recommandation : Supprimer ces utilisateurs du code standard et isoler correctement les tests de l'application.

B. Couverture de Tests & Qualité du Code (Niveau de Priorité : P1)

Le rapport JaCoCo indique une couverture globale de 70 % des instructions, mais révèle une forte disparité selon les packages :

- Package config (Zone fragile) : Couverture d'instructions à 44 % et 0 % des branches exécutées (4 branches manquées sur 4). C'est une zone aveugle critique car c'est là que réside la configuration de sécurité.
- Package model : Couverture d'instructions à 58 %.
- Packages solides : service (92 % couvert), controller (100 %) et dto (100 %).
- Duplication : 0.0 % sur 509 lignes. Le code est propre, bien découpé et sans redondance.

3. Plan d'Action

P0 : Correctifs Sécurité Immédiat

1. Remplacement du Cipher : Refactoriser le composant cryptographique pour appliquer du AES/GCM/NoPadding ou un algorithme conforme aux normes OWASP actuelles.
2. Isolation des utilisateurs de tests dans le bon contexte et suppression de l'effet de bord métier.
3. Re-scan SonarQube : Valider le passage de la note Sécurité de D → A.

P1 : Sécurisation par les Tests (Sprint Refactoring - 1 semaine)

1. Sécuriser la Configuration (com.gatcha.auth_api.config) : Rédiger des tests d'intégration (via @SpringBootTest) pour faire passer la couverture du package de 44 % à +80 % et tester les branches logiques à 100 %.
2. Compléter le Package model : Ajouter les tests unitaires manquants pour remonter la couverture à >80 %.

4. Proposition d'Intervention

Je vous propose de prendre en charge la correction des points abordés dans ce rapport sous la forme d'un Sprint de Modernisation et Sécurisation :

- Livrables :
 - Correction du chiffrement, de l'effet de bord et validation SonarQube (Note Sécurité à A).
 - Ajout des suites de tests unitaires/intégration JUnit 5 avec isolation totale du contexte de test de l'application. Couverture globale ramenée à >80 %).
 - Rapport final de contre-expertise validant la livraison.
- Forfait proposé : 850 € HT.